КУРСОВАЯ РАБОТА

по дисциплине «Объектно-ориентированное программирование»

Тема: Разработка Telegram-бота для мониторинга суточного потребления калорий

Автор: Кутепов П.О.

Санкт-Петербург

2025

АННОТАЦИЯ

В работе представлена разработка Telegram-бота для автоматизированного расчёта индивидуальных суточных норм калорий и макронутриентов (КБЖУ) на основе данных и цели пользователя. Реализованы функции персональной регистрации, учёта потребляемых продуктов и формирования статистики. Бот использует реляционную базу данных SQLite для хранения информации и библиотеку Telebot для взаимодействия с пользователем. Результатом является готовое к использованию приложение для поддержки здорового питания.

СОДЕРЖАНИЕ

АННОТАЦИЯ 2

СОДЕРЖАНИЕ 3

ВВЕДЕНИЕ 4

КРАТКИЕ СВЕДЕНИЯ О РАСЧЕТЕ НОРМ КБЖУ 5

1.1. Определение дневной нормы калорий 5

1.2. Определение норм БЖУ 5

ПРОГРАММНАЯ РЕАЛИЗАЦИЯ 7

2.1. Некоторые теоретические сведения 7

2.2. Диаграмма вариантов использования 8

2.3. Диаграмма классов объектной модели предметной области 9

2.4. Спецификация классов 10

2.5. Описание интерфейса пользователя программы 17

ЗАКЛЮЧЕНИЕ 19

ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ 20

ВВЕДЕНИЕ

В условиях растущего интереса к здоровому образу жизни, автоматизированный учёт потребляемых калорий и макронутриентов (белков, жиров, углеводов) становится важным инструментом для достижения фитнес-целей и поддержания здоровья. Telegram, благодаря своей популярности и открытому API, предоставляет удобную платформу для создания интерактивных сервисов, в том числе персональных помощников по питанию. Целью данной работы является разработка Telegram-бота, который предоставит пользователям персонализированные нормы питания на основе их физиологических данных и целей. Для этого требуется теоретическое обоснование алгоритмов расчёта, проектирование базы данных для хранения информации о пользователях и их рационе с использованием SQLite, а также реализация диалогового интерфейса с помощью библиотеки Telebot для Python.

КРАТКИЕ СВЕДЕНИЯ О РАСЧЕТЕ НОРМ КБЖУ

1.1. Определение дневной нормы калорий

Базальный метаболизм (BMR) — минимальное количество калорий, которое организм тратит в состоянии полного покоя, чтобы поддерживать жизненно важные функции: дыхание, кровообращение, работу мозга, сердца, почек и других органов, а также регуляцию температуры тела. Для расчета BMR рекомендуется использовать уравнение Миффлина-Сан Жеора:

$\text{BMR}_{\text{для мужчин}} = (10 \times \text{вес в кг}) + (6.25 \times \text{рост в см}) - (5 \times \text{возраст в годах}) + 5$

$BMR_{\text{для женщин}} = (10 \times \text{вес в кг}) + (6.25 \times \text{рост всм}) - (5 \times \text{возраст в годах}) - 161.$

Суточная норма калорий определяется умножением BMR на коэффициенты физической активности и корректируется под конкретные цели человека (табл. 1-2) [1].

Тип физической активности	**Коэффициент**
Сидячий образ жизни	1.2
Лёгкая активность (1-3 тренировки в неделю)	1.375
Умеренная активность (3-5 тренировок)	1.55
Высокая активность (6-7 тренировок)	1.725
Очень высокая активность (тяжёлая физическая работа и тренировки)	1.9

Таблица 1 – Коэффициенты физической активности

Цель	**Изменение BMR**
Для похудения	Минус 10-20% от нормы
Для набора мышечной массы	Профицит в 5-15%
Для поддержания веса	На уровне нормы

Таблица 2 – Корректировка нормы калорий в зависимости от цели

1.2. Определение норм БЖУ

Количество белка в день определяется по массе человека в зависимости от его цели (Табл. 3) [2]. Жиры обычно составляют 25% от

дневной нормы калорий. Углеводы рассчитываются по остаточному принципу по уравнению:

$$\text{калории} = 4 \times \text{белки} + 9 \times \text{жиры} + 4 \times \text{углеводы}.$$ [3]

Цель	**Количество гр/кг**
Для похудения	1.6-2
Для набора мышечной массы	1.6-2.4
Для поддержания веса	1.2-1.6

Таблица 3 – Количество белка на килограмм массы тела

ПРОГРАММНАЯ РЕАЛИЗАЦИЯ

2.1. Некоторые теоретические сведения

При написании бота, в основе лежит идея обработчиков (handlers). Это функции, которые срабатывают на определённые действия пользователя: на команду /start, на текст или на нажатие кнопки. Основные инструменты для общения с пользователем — это send_message (отправить новое сообщение), edit_message_text (изменить уже отправленное сообщение) и delete_message (удалить сообщение). Чтобы сделать интерфейс удобным, используют inline-кнопки — это кнопки, встроенные в сообщение. Когда пользователь нажимает на кнопку, бот обрабатывает callback и выполняет нужное действие: например, меняет текст сообщения на следующую страницу меню или добавляет выбранный продукт в дневник питания. [4]

SQLite — это база данных, которая хранится локально в одном файле. Для работы с ней, сначала нужно создать таблицу при помощи CREATE TABLE. Для добавления новой записи используют команду INSERT INTO. Чтобы получить данные из таблицы, используют SELECT. Чтобы удалить запись, нужна команда DELETE FROM. Команды отправляются из программы на Python, SQLite выполняет и сохраняет результат в файл. [5]

2.2. Диаграмма вариантов использования

После регистрации пользователь может записать свой приём пищи: либо выбрать продукты из списка, либо добавить продукт вручную, указав пищевую ценность. В любой момент можно посмотреть историю питания или получить статистику за месяц (рис. 1).

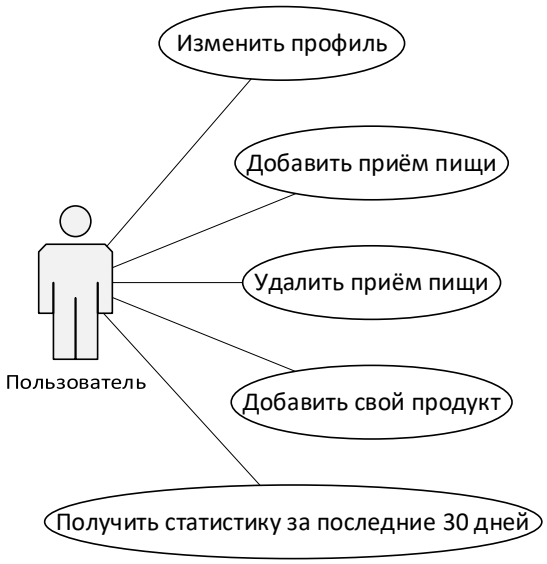


Рисунок 1

2.3. Диаграмма классов объектной модели предметной области

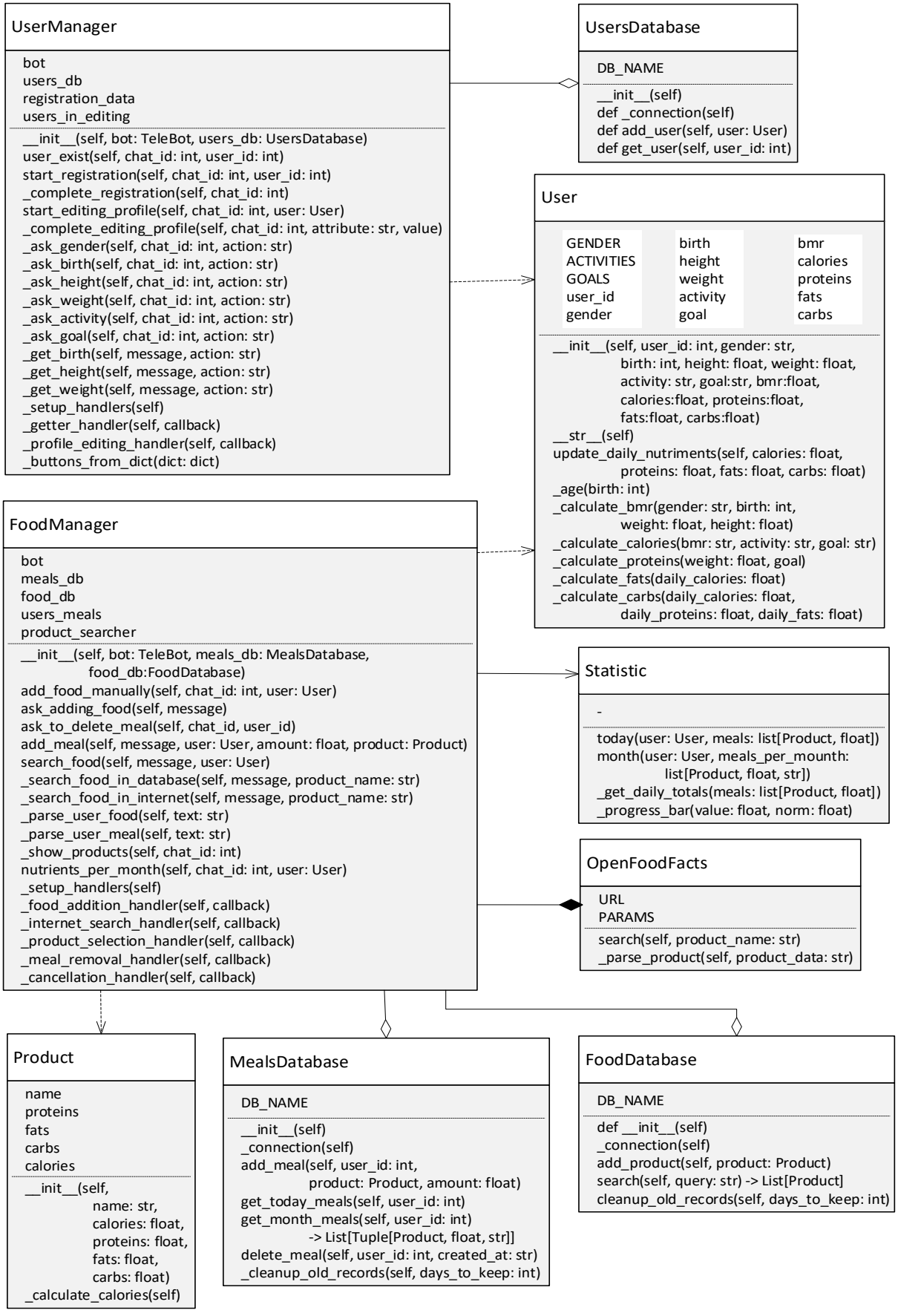


Рисунок 2 – Диаграмма классов

2.4. Спецификация классов

UserManager:

Поля	**Описание полей**
bot: Telebot	указатель на Telegram-бота
users_db: UsersDatabase	База для хранения пользователей
registration_data: dict{dict}	Словарь для временного хранения информации о пользователе
users_in_editing: dict{User}	Словарь пользователей, которые редактируют профиль
Методы	Описание методов
__init__(self, bot: telebot.TeleBot, users_db: UsersDatabase)	Инициализация
user_exist(self, chat_id: int, user_id: int) -> User	Проверяет, есть ли пользователь в базе
start_registration(self, chat_id: int, user_id: int)	Начало регистрации
_complete_registration(self, chat_id: int)	Завершение регистрации, сохранение пользователя в базу
start_editing_profile(self, chat_id: int, user: User)	Начало редактирования профиля, спрашивает, какое поле изменить
_complete_editing_profile(self, chat_id: int, attribute: str, value)	Сохранение пользователя в базу и завершение редактирования профиля
_ask_gender(self, chat_id: int, action: str)	Спрашивает пол у пользователя
_ask_birth(self, chat_id: int, action: str)	Спрашивает год рождения
_ask_height(self, chat_id: int, action: str)	Спрашивает рост

_ask_weight(self, chat_id: int, action: str)	Спрашивает вес
_ask_activity(self, chat_id: int, action: str)	Спрашивает уровень физической активности
_ask_goal(self, chat_id: int, action: str)	Спрашивает цель
_get_birth(self, message, action: str)	Получает от пользователя год рождения
_get_height(self, message, action: str)	Получает рост
_get_weight(self, message, action: str)	Получает вес
_setup_handlers(self)	Инициализирует обработчики inline-кнопок
_getter_handler(self, callback)	Получает информацию с кнопок для регистрации/изменения профиля
_profile_editing_handler(self, callback)	Перенаправляет на функции спроса информации о пользователе
_buttons_from_dict(dict: dict)	Создаёт inline-кнопки по словарю ({button_text: callback})

Таблица 4

User:

Поля	**Описание полей**
user_id: int	id пользователя
gender: str	пол
birth: int	год рождения
height: float	рост
weight: float	вес
activity: str	активность
goal: str	цель
bmr: float	Базальный метаболизм

calories: float	Дневная норма калорий
proteins: float	Дневная норма белков
fats: float	Дневная норма жиров
carbs: float	Дневная норма углеводов
Методы	Описание методов
__init__(self, user_id: int, gender: str, birth: int, height: float, weight: float, activity: str, goal:str, bmr:float=None, calories:float=None, proteins:float=None, fats:float=None, carbs:float=None)	Инициализация
update_daily_nutriments(self, calories:float=None, proteins:float=None, fats:float=None, carbs:float=None)	Обновляет суточные нормы питательных веществ и калорий
_age(birth: int)	Возраст пользователя
_calculate_bmr(gender: str, birth: int, weight: float, height: float)	Считает базальный метаболизм
_calculate_calories(bmr: str, activity: str, goal: str)	Считает дневную норму калорий
_calculate_proteins(weight: float, goal)	Считает дневную норму белков
_calculate_fats(daily_calories: float)	Считает дневную норму жиров
_calculate_carbs(daily_calories: float, daily_proteins: float, daily_fats: float)	Считает дневную норму углеводов

Таблица 5

UserDatabase:

Поля	**Описание полей**
DB_NAME: str	Название файла базы данных
Методы	Описание методов

__init__(self)	Инициализация
_connection(self)	Устанавливает соединение с базой
add_user(self, user: User) -> bool	Сохранение пользователя в базу
get_user(self, user_id: int) -> User	Возвращает пользователя из базы

Таблица 6

FoodManager:

Поля	**Описание полей**
bot: TeleBot	указатель на бота
meals_db: MealsDatabase	База данных с приёмами пищи
food_db: FoodDatabase	Локальная база данных продуктами
users_meals: dict{list[User, float, list[Product]]}	Временное хранилище записи приёма пищи
product_searcher: OpenFoodFacts	Для поиска продуктов на сторонней базе
Методы	Описание методов
__init__(self, bot: telebot.TeleBot, meals_db: MealsDatabase, food_db:FoodDatabase)	Инициализация
add_food_manually(self, chat_id: int, user: User)	Ручное добавление приёма пищи (вместе с названием продукта указывается его КБЖУ)
ask_adding_food(self, message)	Спрашивает, какую еду из списка добавить
ask_to_delete_meal(self, chat_id, user_id)	Спрашивает, какую запись приёма пищи удалить из базы
add_meal(self, message, user: User, amount: float, product: Product)	Сохраняет приём пищи

search_food(self, message, user: User)	Ищет еду на локальной или сторонней базе
_search_food_in_database(self, message, product_name: str)	Ищет продукты на локальной базе
_search_food_in_internet(self, message, product_name: str)	Ищет еду на сторонней базе
_parse_user_food(self, text: str) -> Tuple[Optional[Product], Optional[float]]	Парсинг названия продукта, его КБЖУ и съеденного количества
_parse_user_meal(self, text: str) -> Tuple[Optional[str], Optional[float]]	Парсинг названия продукта и съеденного количества
_show_products(self, chat_id: int)	Возвращает найденные продукты inline-кнопками для выбора
nutrients_per_month(self, chat_id: int, user: User)	Выводит статистику в виде графиков за месяц
_setup_handlers(self)	Инициализирует обработчики inline-кнопок
_food_addition_handler(self, callback)	Срабатывает, когда пользователь подтверждает сохранение продукта
_internet_search_handler(self, callback)	Перенаправляет на функцию для поиска продуктов на сторонней базе
_product_selection_handler(self, callback)	Обрабатывает выбор конкретного продукта пользователем
_meal_removal_handler(self, callback)	Обрабатывает выбор удаляемой записи из БД пользователем
_cancellation_handler(self, callback)	Обрабатывает кнопку отмены

Таблица 7

Product:

Поля	**Описание полей**
name: str	Название продукта
proteins: float	Содержание белков на 100 гр.
fats: float	Жиров на 100 гр.
carbs: float	Углеводов на 100 гр.
calories: float	Калорийность на 100 гр.
Методы	Описание методов
__init__(self, name: str, calories: float, proteins: float, fats: float, carbs: float)	Инициализация
_calculate_calories(self)	Расчет калорийности продукта на 100 гр.

Таблица 8

MeaksDatabase:

Поля	**Описание полей**
DB_NAME: str	Название файла БД
Методы	Описание методов
__init__(self)	Инициализация
_connection(self)	Подключение к базе
add_meal(self, user_id: int, product: Product, amount: float) -> bool	Сохраняет приём пищи
get_today_meals(self, user_id: int) -> List[Tuple[Product, float, str]]	Возвращает все приёмы пищи за день
get_month_meals(self, user_id: int) -> List[Tuple[Product, float, str]]	Возвращает все приёмы пищи за месяц
delete_meal(self, user_id: int, created_at: str) -> bool	Удаляет приём пищи по дате
_cleanup_old_records(self, days_to_keep: int = 31)	Очистка старых записей (которым больше месяца)

Таблица 9

FoodDatabase:

Поля	**Описание полей**
DB_NAME: str	Название файла БД
Методы	Описание методов
__init__(self)	Инициализация
_connection(self)	Подключение к базе
add_product(self, product: Product) -> bool	Сохранение продукта в базу
search(self, query: str) -> List[Product]	Поиск продуктов в базе
cleanup_old_records(self, days_to_keep: int = 90) -> int	Очистка старых записей (которым больше 90 дней)

Таблица 10

Statistic:

Методы	**Описание методов**
today(user: User, meals: list[Product, float])	Статистика на сегодня
month(user: User, meals_per_mounth: list[Product, float, str])	Статистика за месяц
_get_daily_totals(meals: list[Product, float]) -> dict	Суммирует все КБЖУ за день
_progress_bar(value: float, norm: float) -> str	Рисует прогресс пользователя

Таблица 11

OpenFoodFacts:

Поля	**Описание полей**
URL: str	Url-адрес базы с продуктами
PARAMS: dict{str: str}	Параметры для поиска
Методы	Описание методов

search(self, product_name: str) -> list	Поиск по API
_parse_product(self, product_data: str) -> Product	Парсинг продукта, полученного по API

Таблица 12

2.5. Описание интерфейса пользователя программы

Команда /start, запускает регистрацию. В процессе бот последовательно запрашивает у пользователя данные: пол, год рождения, рост, вес, а также уровень физической активности и поставленную цель (поддержание массы тела, её снижение или увеличение). На основе полученных данных, бот определяет индивидуальные суточные нормы калорий и БЖУ, после чего уведомляет о завершении процесса (рис. 3).

Для сохранения приёма пищи, необходимо отправить сообщение, содержащее наименование продукта и потреблённое количество (в граммах), разделённые пробелом или запятой (рис. 4).

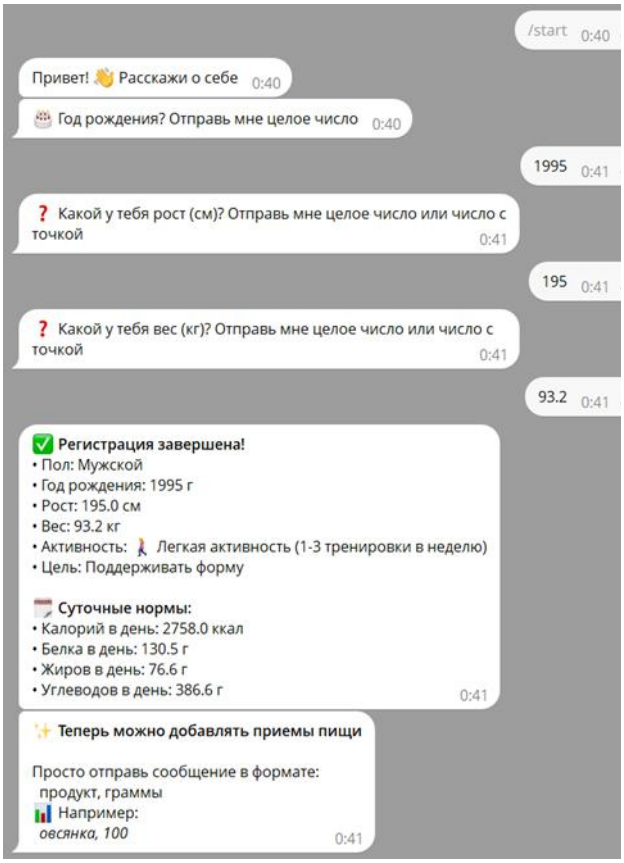


Рисунок 3 - Процесс регистрации



Рисунок 4 - Добавление продукта

В случае отсутствия продукта в базе, реализован механизм ручного добавления. Для этого пользователь вводит данные в формате: каждая строка должна содержать наименование нового продукта, числовые значения его калорийности и содержания макронутриентов (белки, жиры, углеводы), а также потреблённое количество, разделённые пробелами (рис. 5).

Каждое успешное добавление завершается выводом сводной статистики за текущий день, отражающей степень выполнения норм. Дополнительно по запросу пользователя формируются и отображаются графики изменения суточных показателей КБЖУ за 30 дней (рис. 6).

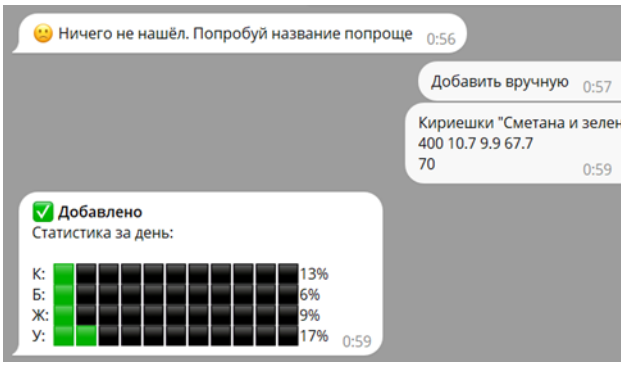


Рисунок 5 - Добавление вручную

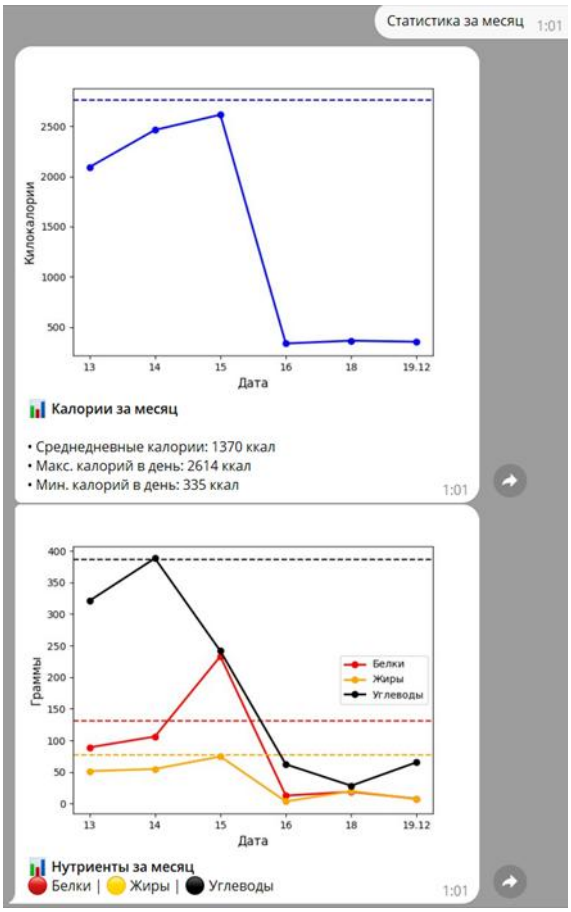


Рисунок 6 - Статистика за месяц

ЗАКЛЮЧЕНИЕ

Целью работы была разработка Telegram-бота для расчёта индивидуальных норм КБЖУ и ведения пищевого дневника. Для её достижения были изучены и применены методики нутрициологического расчёта, спроектированы структуры баз данных SQLite и освоен API Telegram посредством библиотеки Telebot. В результате создано работоспособное приложение, позволяющее пользователям объективно контролировать свой рацион в удобном формате.

ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Подсчет калорий: как научиться эффективному контролю питания // Физикл, URL: https://fizikl.org/blog/countingofcalories-2/ (дата обращения: 21.11.2025)

2. Белок. Сколько вешать в граммах? // Физикл, URL: https://fizikl.org/blog/belok-skolko-veshat-v-grammah/ (дата обращения: 21.11.2025)

3. Как правильно считать калории? // Spirit.fitness, URL: https://spiritfit.ru/blog/zdorove/kak-pravilno-schitat-kalorii (дата обращения: 21.11.2025)

4. Разработка Telegram ботов на Python // Itproger, URL: https://youtube.com/playlist?list=PL0lO_mIqDDFUev1gp9yEwmwcy8SicqKbt&si=I_8t9xLIy5jAptqt (дата обращения: 29.11.2025)

5. Руководство по SQLite в Python // PythonRu, URL: https://pythonru.com/osnovy/sqlite-v-python (дата обращения: 4.12.2025)